Proyecto de organización de archivos usando bash. Semana Tec

Miguel A01645430

Pablo A01644172

Jesús A01648970



29 de octubre de 2026

El problema

- Nos dieron un zip de 47 MB con 99 archivos revueltos en una sola carpeta.
- Adentro había 60 archivos .py, csv del Titanic, un .ttf, un .mp3, imágenes, y un archivo llamado tic_tac_toe sin extensión.
- La pregunta del proyecto: ¿cuáles de esos .py usan PySide6? Contestarla a ojo era la opción lenta.
- Y en GitHub la carpeta se convirtió en zip a medio proyecto. Eso también había que resolverlo.



El zip, más o menos así.

¿Y a mano?

Se podía. También se puede ordenar el cuarto.

- 99 archivos a mano son 99 veces copiar, pegar y elegir carpeta.
- 60 archivos .py abiertos uno por uno para buscar PySide6 con los ojos.
- Y si un nombre se repite, adiós al original.
- Un script hace todo eso en segundos y no se cansa.



Nosotros, pensando en hacerlo a mano.

El enfoque

Un script de bash con dos argumentos: el zip y la carpeta destino.

```
proyecto_analisis.zip
         |
         v
 [ carpeta temporal ]   <-- el zip original no se toca
         |
         +--> documentos/
         +--> imagenes/
         +--> codigo/  ------>  auditoría de PySide6
         +--> multimedia/
         +--> otros/
```

- El zip se descomprime aparte y de ahí se copia todo. El original queda intacto, con el mismo MD5.
- Cada extensión tiene su carpeta. Lo que no encaja cae en otros/.
- Nada se sobrescribe: si ya existe logo.png, el nuevo se guarda como logo_1.png.
- Bash ya viene en macOS. No hubo que instalar nada.

Cómo funciona

1. Revisa que lleguen dos argumentos y que el zip exista. Si no, imprime el uso y sale con `exit 1`.
2. Crea una carpeta temporal con `mktemp -d` y descomprime ahí con `unzip -q`.
3. Recorre los archivos con `find` y decide la carpeta con `case "$EXTENSION"`.
4. Audita cada .py con `grep -n -i "PySide6"` y muestra línea y texto.
5. Suma los contadores con `$(( ))` y escribe el resumen final.

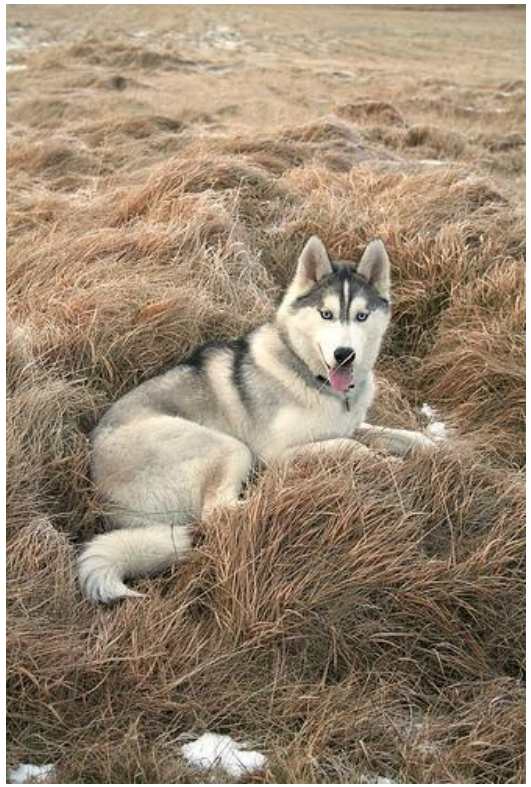
```
$ ./script_miguel.sh proyecto_analisis.zip resultado
  QT_image.py  ->  codigo/

Archivo: resultado/codigo/QT_GIF.py
  SI usa PySide6. Coincidencias (linea: texto):
  2:from PySide6.QtGui import QMovie
```

Sí, el script es básico. También funciona, que era el punto.

GitHub y el trabajo en equipo

- El repo en la nube desde el primer día: git clone y a trabajar.
- Cada quien en su rama para no pisarse: miguel-2, pablo y jesus.
- Commits chiquitos, git push seguido y git pull antes de subir.
- El commit del recuerdo: "quitar carpeta y poner zip". 100 archivos cambiados y 129,272 líneas borradas.
- El zip pesa 45.14 MiB y GitHub lo aceptó. El límite son 100 MB, así que pasamos con suspenso.
- Al final todo quedó en main, sin conflictos y con el proyecto completo.



El equipo después del merge.

Y aquí entra DeepSeek

Le pedimos el script completo de una vez, con un prompt que dejaba las reglas claras: bash de principiante, comentarios en español y nada de trucos elegantes.

```
"Crea un script de Bash llamado script_miguel.sh... recibe un zip y una
carpeta destino, descomprime en una temporal, clasifica por extensión,
audita los .py buscando PySide6 con grep -n, y muestra un resumen.
Soy principiante: usa if, case, for, while, grep. Nada de arrays,
awk ni set -e."
```

Salieron unas 200 líneas comentadas que entiende cualquiera que sepa if, for y grep. Le pedimos que no fuera elegante y cumplió.



DeepSeek, en su oficina.

Resultados

99
archivos ordenados

60
archivos .py

28
usan PySide6

5
carpetas

- El zip original conserva su MD5 (800e5b40…), comprobado antes y después de correr el script.
- `La auditoría imprime archivo, línea y texto de cada coincidencia, sin abrir nada a mano.`
- Correr el script completo toma segundos.

Gracias. ¿Preguntas?